

Python datetime module Tasks

```
from datetime import datetime, date, time, timedelta, timezone
```

```
# TASK 1 – Get the current date and time
# Task:
# Print the current date and time.
```

```
# Solution:
now = datetime.now()

print("Current datetime:", now)
```

```
# TASK 2 – Get today's date
# Task:
# Print today's date only.
```

```
# Solution:
today = date.today()

print("Today's date:", today)
```

```
# TASK 3 – Create a specific datetime
# Task:
# Create a datetime representing:
# 2026-09-07 14:30:00
```

```
# Solution:
dt = datetime(2026, 9, 7, 14, 30, 0)

print("Specific datetime:", dt)
```

```
# TASK 4 – Extract datetime components
# Task:
# From the datetime below, extract:
# - year
# - month
# - day
# - hour
# - minute
# - second
```

```
dt = datetime(2026, 9, 7, 14, 30, 45)
```

```
# Solution:
print("Year:", dt.year)
print("Month:", dt.month)
print("Day:", dt.day)
print("Hour:", dt.hour)
print("Minute:", dt.minute)
print("Second:", dt.second)
```

```
# TASK 5 – Create a date
# Task:
# Create a date for January 15, 2026.
```

```
# Solution:
my_date = date(2026, 1, 15)

print("Date:", my_date)
```

```

# TASK 6 – Create a time
# Task:
# Create a time representing 08:30:15.

# Solution:
my_time = time(8, 30, 15)

print("Time:", my_time)

# TASK 7 – Add days to a date
# Task:
# Add 7 days to:
# 2026-09-07

start_date = date(2026, 9, 7)

# Solution:
new_date = start_date + timedelta(days=7)

print("Original date:", start_date)
print("After 7 days:", new_date)

# TASK 8 – Subtract days from a datetime
# Task:
# Subtract 3 days from:
# 2026-09-07 10:00:00

dt = datetime(2026, 9, 7, 10, 0, 0)

# Solution:
new_dt = dt - timedelta(days=3)

print("Original:", dt)
print("3 days earlier:", new_dt)

# TASK 9 – Add hours and minutes
# Task:
# Add 5 hours and 30 minutes to:
# 2026-09-07 08:00:00

dt = datetime(2026, 9, 7, 8, 0, 0)

# Solution:
new_dt = dt + timedelta(hours=5, minutes=30)

print("Original:", dt)
print("New datetime:", new_dt)

# TASK 10 – Calculate the difference between two dates
# Task:
# Calculate how many days are between:
# 2026-09-01
# 2026-09-07

date1 = date(2026, 9, 1)
date2 = date(2026, 9, 7)

# Solution:
difference = date2 - date1

print("Difference:", difference)
print("Number of days:", difference.days)

```

```

# TASK 11 – Calculate datetime difference
# Task:
# Calculate the duration between:
# 08:30:00
# 12:45:30

start = datetime(2026, 9, 7, 8, 30, 0)
end = datetime(2026, 9, 7, 12, 45, 30)

# Solution:
duration = end - start

print("Duration:", duration)
print("Seconds:", duration.total_seconds())

# TASK 12 – Compare dates
# Task:
# Determine which date is earlier.

date1 = date(2026, 9, 5)
date2 = date(2026, 9, 7)

# Solution:
if date1 < date2:
    print("date1 is earlier")
elif date1 > date2:
    print("date2 is earlier")
else:
    print("The dates are equal")

# TASK 13 – Format datetime as a string
# Task:
# Convert:
# 2026-09-07 14:30:45
#
# into:
# 2026-09-07 14:30:45

dt = datetime(2026, 9, 7, 14, 30, 45)

# Solution:
formatted = dt.strftime("%Y-%m-%d %H:%M:%S")

print("Formatted:", formatted)

# TASK 14 – Format datetime for a filename
# Task:
# Create a timestamp suitable for a filename.
#
# Example:
# events_20260907_143045.json

dt = datetime(2026, 9, 7, 14, 30, 45)

# Solution:
timestamp = dt.strftime("%Y%m%d_%H%M%S")

filename = f"events_{timestamp}.json"

print("Filename:", filename)

```

```

# TASK 15 – Convert string to datetime
# Task:
# Convert this string into a datetime object:
#
# "2026-09-07 14:30:45"

date_string = "2026-09-07 14:30:45"

# Solution:
dt = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")

print("Datetime:", dt)
print("Type:", type(dt))

# TASK 16 – Parse an ISO datetime
# Task:
# Convert this ISO-format string into a datetime:
#
# "2026-09-07T14:30:45"

date_string = "2026-09-07T14:30:45"

# Solution:
dt = datetime.fromisoformat(date_string)

print("Datetime:", dt)

# TASK 17 – Check if data is older than 1 hour
# Data engineering scenario:
#
# An event arrived at:
# 2026-09-07 10:00:00
#
# Current time:
# 2026-09-07 11:30:00
#
# Determine whether the event is older than 1 hour.

event_time = datetime(2026, 9, 7, 10, 0, 0)
current_time = datetime(2026, 9, 7, 11, 30, 0)

# Solution:
age = current_time - event_time

if age > timedelta(hours=1):
    print("Event is older than 1 hour")
else:
    print("Event is recent")

# TASK 18 – Find records from the last 24 hours
# Data engineering scenario:
#
# Current time:
# 2026-09-07 12:00:00
#
# Event time:
# 2026-09-06 15:00:00
#
# Determine whether the event occurred within the last 24 hours.

current_time = datetime(2026, 9, 7, 12, 0, 0)
event_time = datetime(2026, 9, 6, 15, 0, 0)

```

```

# Solution:
cutoff = current_time - timedelta(hours=24)

if event_time >= cutoff:
    print("Event is within the last 24 hours")
else:
    print("Event is older than 24 hours")


# TASK 19 – Calculate event processing delay
# Data engineering scenario:
#
# Event happened at:
# 10:00:00
#
# Data was processed at:
# 10:02:30
#
# Calculate the processing delay.

event_time = datetime(2026, 9, 7, 10, 0, 0)
processing_time = datetime(2026, 9, 7, 10, 2, 30)

# Solution:
delay = processing_time - event_time

print("Processing delay:", delay)
print("Delay in seconds:", delay.total_seconds())


# TASK 20 – Create daily dates
# Data engineering scenario:
#
# Generate 7 dates starting from:
# 2026-09-01

start_date = date(2026, 9, 1)

# Solution:
for i in range(7):
    current_date = start_date + timedelta(days=i)
    print(current_date)


# TASK 21 – Generate hourly timestamps
# Data engineering scenario:
#
# Generate 5 timestamps starting from:
# 2026-09-07 08:00:00
#
# Each timestamp should be 1 hour apart.

start_time = datetime(2026, 9, 7, 8, 0, 0)

# Solution:
for i in range(5):
    timestamp = start_time + timedelta(hours=i)
    print(timestamp)


# TASK 22 – Extract date from datetime
# Task:
# Extract only the date from a datetime.

dt = datetime(2026, 9, 7, 14, 30, 45)

```

```

# Solution:
only_date = dt.date()

print("Date:", only_date)

# TASK 23 – Extract time from datetime
# Task:
# Extract only the time from a datetime.

dt = datetime(2026, 9, 7, 14, 30, 45)

# Solution:
only_time = dt.time()

print("Time:", only_time)

# TASK 24 – Get the day of the week
# Task:
# Determine the weekday number.
#
# Monday = 0
# Sunday = 6

dt = datetime(2026, 9, 7)

# Solution:
print("Weekday number:", dt.weekday())

# TASK 25 – Get weekday name
# Task:
# Print the weekday name.

dt = datetime(2026, 9, 7)

# Solution:
weekday_name = dt.strftime("%A")

print("Weekday:", weekday_name)

# TASK 26 – Create UTC datetime
# Task:
# Create a UTC datetime representing:
# 2026-09-07 12:00:00

# Solution:
utc_dt = datetime(
    2026,
    9,
    7,
    12,
    0,
    0,
    tzinfo=timezone.utc
)

print("UTC datetime:", utc_dt)

```

```
# TASK 27 – Check timezone awareness
# Task:
# Determine whether these datetimes are timezone-aware.
```

```
naive_dt = datetime(2026, 9, 7, 12, 0, 0)
```

```
aware_dt = datetime(
    2026,
    9,
    7,
    12,
    0,
    0,
    tzinfo=timezone.utc
)
```

```
# Solution:
print("Naive:", naive_dt.tzinfo)
print("Aware:", aware_dt.tzinfo)
```

```
# TASK 28 – Unix timestamp
# Data engineering scenario:
#
# Convert a datetime into a Unix timestamp.
```

```
dt = datetime(
    2026,
    9,
    7,
    12,
    0,
    0,
    tzinfo=timezone.utc
)
```

```
# Solution:
timestamp = dt.timestamp()
print("Unix timestamp:", timestamp)
```

```
# TASK 29 – Convert Unix timestamp to datetime
# Task:
# Convert a Unix timestamp back into a UTC datetime.
```

```
timestamp = 1788782400
```

```
# Solution:
dt = datetime.fromtimestamp(timestamp, tz=timezone.utc)
print("Datetime:", dt)
```

```
# TASK 30 – DATA ENGINEERING MINI TASK
# You receive event timestamps as strings:
#
# "2026-09-07 08:00:00"
# "2026-09-07 09:15:00"
# "2026-09-07 10:30:00"
#
# Convert them into datetime objects and calculate
# the difference between each event and the first event.
```

```
event_strings = [
    "2026-09-07 08:00:00",
    "2026-09-07 09:15:00",
    "2026-09-07 10:30:00"
]
```

```
# Solution:
events = [
    datetime.strptime(event, "%Y-%m-%d %H:%M:%S")
    for event in event_strings
]

first_event = events[0]

for event in events:
    difference = event - first_event

    print(
        event,
        "->",
        difference
    )
```